In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
import scipy
```

In [30]:
```
df=pd.read_csv("StudentPerformance.csv")
df
```

Out[30]:

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Scores	Motivati
0	23	84	Low	High	No	7	73	
1	19	64	Low	Medium	No	8	59	
2	24	98	Medium	Medium	Yes	7	91	
3	29	89	Low	Medium	Yes	8	98	
4	19	92	Medium	Medium	Yes	6	65	
...	...	...	...	...	...	...	...	
6602	25	69	High	Medium	No	7	76	
6603	23	76	High	Medium	No	8	81	
6604	20	90	Medium	Low	Yes	6	65	
6605	10	86	High	High	Yes	6	91	
6606	15	67	Medium	Low	Yes	9	94	

6607 rows × 20 columns

steps1 Business problem undersranding

In [4]:
```
# To improve the Student performance in the examination
```

In [ ]:
```
# DOMAIN:-School Dataset
```

In [ ]:
```
#Description:-
# This project aims to identify key factors affecting student exam performance using data analysis.
#It helps suggest ways to support students and improve their academic results
```

steps2 data understanding and exploration

In [5]:
```
df.head()
```

Out[5]:

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Scores	Motivation_L
0	23	84	Low	High	No	7	73	
1	19	64	Low	Medium	No	8	59	
2	24	98	Medium	Medium	Yes	7	91	Me
3	29	89	Low	Medium	Yes	8	98	Me
4	19	92	Medium	Medium	Yes	6	65	Me

In [ ]:
```
# observe:- by default we see top 5 rows
```

In [7]:
```
df.tail()
```

Out[7]:

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Scores	Motivati
6602	25	69	High	Medium	No	7	76	
6603	23	76	High	Medium	No	8	81	
6604	20	90	Medium	Low	Yes	6	65	
6605	10	86	High	High	Yes	6	91	
6606	15	67	Medium	Low	Yes	9	94	

In [ ]:
```
# observe:- by default we see last 5 rows
```

In [11]:
```
df.shape
```

Out[11]:
```
(6607, 20)
```

In [ ]:
```
# observe:-i observe that there are 6607 rows and 20 columns
```

In [13]:
```
df.size
```

Out[13]:
```
132140
```

In [ ]:
```
# observe:- i observe that there are 132140 data available  in this Dataset
```

In [15]:
```
df.dtypes
```

Out[15]:
```
Hours_Studied                int64
Attendance                   int64
Parental_Involvement        object
Access_to_Resources         object
Extracurricular_Activities  object
Sleep_Hours                  int64
Previous_Scores              int64
Motivation_Level            object
Internet_Access             object
Tutoring_Sessions            int64
Family_Income               object
Teacher_Quality             object
School_Type                 object
Peer_Influence              object
Physical_Activity            int64
Learning_Disabilities       object
Parental_Education_Level    object
Distance_from_Home          object
Gender                      object
Exam_Score                   int64
dtype: object
```

In [21]:
```
discrete=df.select_dtypes(include="object").columns
discrete.tolist()
```

Out[21]:
```
['Parental_Involvement',
 'Access_to_Resources',
 'Extracurricular_Activities',
 'Motivation_Level',
 'Internet_Access',
 'Family_Income',
 'Teacher_Quality',
 'School_Type',
 'Peer_Influence',
 'Learning_Disabilities',
 'Parental_Education_Level',
 'Distance_from_Home',
 'Gender']
```

In [25]:
```
c=df.select_dtypes(include="int")
c
```

Out[25]:

	Hours_Studied	Attendance	Sleep_Hours	Previous_Scores	Tutoring_Sessions	Physical_Activity	Exam_Score
0	23	84	7	73	0	3	67
1	19	64	8	59	2	4	61
2	24	98	7	91	2	4	74
3	29	89	8	98	1	4	71
4	19	92	6	65	3	4	70
...	...	...	...	...	...	...	...
6602	25	69	7	76	1	2	68
6603	23	76	8	81	3	2	69
6604	20	90	6	65	3	2	68
6605	10	86	6	91	2	3	68
6606	15	67	9	94	0	4	64

6607 rows × 7 columns

In [ ]:
```
# observe:- here i observe that there are 13 categorical and 7 continuous variable
```

In [23]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6607 entries, 0 to 6606
Data columns (total 20 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Hours_Studied               6607 non-null   int64
 1   Attendance                  6607 non-null   int64
 2   Parental_Involvement        6607 non-null   object
 3   Access_to_Resources         6607 non-null   object
 4   Extracurricular_Activities  6607 non-null   object
 5   Sleep_Hours                 6607 non-null   int64
 6   Previous_Scores             6607 non-null   int64
 7   Motivation_Level            6607 non-null   object
 8   Internet_Access             6607 non-null   object
 9   Tutoring_Sessions           6607 non-null   int64
 10  Family_Income               6607 non-null   object
 11  Teacher_Quality             6529 non-null   object
 12  School_Type                 6607 non-null   object
 13  Peer_Influence              6607 non-null   object
 14  Physical_Activity           6607 non-null   int64
 15  Learning_Disabilities       6607 non-null   object
 16  Parental_Education_Level    6517 non-null   object
 17  Distance_from_Home          6540 non-null   object
 18  Gender                      6607 non-null   object
 19  Exam_Score                  6607 non-null   int64
dtypes: int64(7), object(13)
memory usage: 1.0+ MB
```

In [ ]:
```
# Obesreve:  there are some missing value in  Distance_from_Home , Teacher_Quality column
# and Parental_Education_Level.
```

In [15]:
```
df.isnull().sum()
```

Out[15]:
```
Hours_Studied                  0
Attendance                     0
Parental_Involvement           0
Access_to_Resources            0
Extracurricular_Activities     0
Sleep_Hours                    0
Previous_Scores                0
Motivation_Level               0
Internet_Access                0
Tutoring_Sessions              0
Family_Income                  0
Teacher_Quality               78
School_Type                    0
Peer_Influence                 0
Physical_Activity              0
Learning_Disabilities          0
Parental_Education_Level      90
Distance_from_Home            67
Gender                         0
Exam_Score                     0
dtype: int64
```

In [ ]:
```
# observe : there are three columns have null value .
# in  Teacher_Quality column(78 missing value approxmately 1.2% of the Data )
# in Parental_Education_Level column(90 missing value approxmately 1.4 % of the Data)
# and in Distance_from_Home  column(67 missing value approxmately 1 % of the Data )
```

In [19]:
```
df.duplicated().sum()
```

Out[19]:
```
0
```

In [ ]:
```
# Obesreve: there is no duplicates value available in this Dataset
```

In [21]:
```
df.columns.tolist()
```

Out[21]:
```
['Hours_Studied',
 'Attendance',
 'Parental_Involvement',
 'Access_to_Resources',
 'Extracurricular_Activities',
 'Sleep_Hours',
 'Previous_Scores',
 'Motivation_Level',
 'Internet_Access',
 'Tutoring_Sessions',
 'Family_Income',
 'Teacher_Quality',
 'School_Type',
 'Peer_Influence',
 'Physical_Activity',
 'Learning_Disabilities',
 'Parental_Education_Level',
 'Distance_from_Home',
 'Gender',
 'Exam_Score']
```

In [ ]:
```
#explain each column
#Hours studied: Number of hours a student studies per week.
#Attendance: Percentage of classes the student has attended.
#Parental Involvement: Level of parental support in education (Low, Medium, High).
#Access_to_Resource: Availability of educational resources (Low, Medium, High).
#Extracurricular Activities: Participation in extracurricular activities (Yes, No).
#Sleep Hours: Average number of hours a student sleeps per night.
#Previous Scores: Student’s scores from earlier exams.
#Motivational level: Student's motivation level (Low, Medium, High).
#Internet access: Whether the student has access to the internet (Yes, No).
#Tutoring Sessions: Number of tutoring sessions attended monthly.
#Family Income: Level of family income (Low, Medium, High).
#Teacher Quality: Quality of teachers (Low, Medium, High).
#School Type: Type of school the student attends (Public, Private).
#Peer Influence: Influence from peers (Positive, Neutral, Negative).
#Physical_Activity: Average hours of physical activity per week.
#Learning Disabilities: Whether the student has any learning disability (Yes, No).
#Parental_Education_Level: Highest education level of parents (High school, College, Postgraduate).
#Distance_From_Home: Distance from home to school (Near, Moderate, Far).
#Gender: Gender of the student (Male, Female).
#Exam Score: Final exam score of the student.
```

explore column wise

Hours_Studied

In [36]:
```
df["Hours_Studied"].unique()
```

Out[36]:
```
array([23, 19, 24, 29, 25, 17, 21,  9, 10, 14, 22, 15, 12, 20, 11, 13, 16,
       18, 31,  8, 26, 28,  4, 35, 27, 33, 36, 43, 34,  1, 30,  7, 32,  6,
       38,  5,  3,  2, 39, 37, 44], dtype=int64)
```

In [ ]:
```
# Observation: Hours_Studied is a continuous features Wide range of values
 # which are related hour of study of a student  .
```

Attendance

In [44]:
```
df["Attendance"].unique()
```

Out[44]:
```
array([ 84,  64,  98,  89,  92,  88,  78,  94,  80,  97,  83,  82,  68,
        60,  70,  75,  99,  74,  65,  62,  91,  90,  66,  69,  72,  63,
        61,  86,  77,  71,  67,  87,  73,  96, 100,  81,  95,  79,  85,
        76,  93], dtype=int64)
```

In [46]:
```
df["Attendance"].nunique()
```

Out[46]:
```
41
```

In [50]:
```
df["Attendance"].value_counts()
```

Out[50]:
```
Attendance
67     190
98     187
76     185
77     184
64     182
94     180
84     175
79     175
91     175
82     173
68     170
69     170
80     169
81     168
96     168
73     168
93     167
72     167
74     165
78     165
61     164
95     163
89     162
71     162
97     161
70     161
65     158
83     157
90     156
88     155
63     155
99     154
92     154
62     152
86     151
87     151
75     149
85     146
66     145
60      87
100     81
Name: count, dtype: int64
```

In [ ]:
```
# Attendance is categorical features indicating the student  present or absent :
#among these 67 students has more attendance.
```

Parental_Involvement

In [27]:
```
df["Parental_Involvement"].unique()
```

Out[27]:
```
array(['Low', 'Medium', 'High'], dtype=object)
```

In [29]:
```
df["Parental_Involvement"].value_counts()
```

Out[29]:
```
Parental_Involvement
Medium    3362
High      1908
Low       1337
Name: count, dtype: int64
```

In [ ]:
```
# Parental_Involvement is categorical features indicating the Parental_Involvement of the student with
#three categories : 'Low', 'Medium', 'High'.
#among these   Medium Parental_Involvement  is the most frequent.
```

Access_to_Resources

In [35]:
```
df["Access_to_Resources"].unique()
```

Out[35]:
```
array(['High', 'Medium', 'Low'], dtype=object)
```

In [37]:
```
df["Access_to_Resources"].value_counts()
```

Out[37]:
```
Access_to_Resources
Medium    3319
High      1975
Low       1313
Name: count, dtype: int64
```

In [ ]:
```
# Access_to_Resources is categorical features indicating the Access_to_Resources of the student with
#three categories : 'Low', 'Medium', 'High'.
#among these   Medium Access_to_Resources  is the most frequent.
```

Extracurricular_Activities

In [117…
```
df["Extracurricular_Activities"].unique()
```

Out[117…
```
array(['No', 'Yes'], dtype=object)
```

In [119…
```
df["Extracurricular_Activities"].value_counts()
```

Out[119…
```
Extracurricular_Activities
Yes    3938
No     2669
Name: count, dtype: int64
```

In [ ]:
```
# Access_to_Resources is categorical features indicating the Access_to_Resources of the student with
#two categories : 'No', 'Yes'.
#among these   Yes Extracurricular_Activities  is the most frequent.
```

Sleep_Hours

In [52]:
```
df["Sleep_Hours"].unique()
```

Out[52]:
```
array([ 7,  8,  6, 10,  9,  5,  4], dtype=int64)
```

In [ ]:
```
# Observation: Sleep_Hours is a continuous features Wide range of values
 # which are related to sleep hours of student.
```

Previous_Scores

In [54]:
```
df["Previous_Scores"].unique()
```

Out[54]:
```
array([ 73,  59,  91,  98,  65,  89,  68,  50,  80,  71,  88,  87,  97,
        72,  74,  70,  82,  58,  99,  84, 100,  75,  54,  90,  94,  51,
        57,  66,  96,  93,  56,  52,  63,  79,  81,  69,  95,  60,  92,
        77,  62,  85,  78,  64,  76,  55,  86,  61,  53,  83,  67],
      dtype=int64)
```

In [ ]:
```
# Observation: Previous_Scores is a continuous features Wide range of values
 # which are related to Previous_Scores of the student.
```

Motivation_Level

In [36]:
```
df["Motivation_Level"].unique()
```

Out[36]:
```
array(['Low', 'Medium', 'High'], dtype=object)
```

In [38]:
```
df["Motivation_Level"].value_counts()
```

Out[38]:
```
Motivation_Level
Medium    3351
Low       1937
High      1319
Name: count, dtype: int64
```

In [ ]:
```
# Motivation_Level is categorical features indicating the Motivation_Level of the student with
#three categories : 'Low', 'Medium', 'High'.
#among these   Medium Motivation_Level  is the most frequent.
```

Internet_Access

In [44]:
```
df["Internet_Access"].unique()
```

Out[44]:
```
array(['Yes', 'No'], dtype=object)
```

In [46]:
```
df["Internet_Access"].value_counts()
```

Out[46]:
```
Internet_Access
Yes    6108
No      499
Name: count, dtype: int64
```

In [ ]:
```
# Internet_Access is categorical features indicating the Internet_Access of the student with
#two categories : 'No', 'Yes'.
#among these   Yes Internet_Access  is the most frequent.
```

Tutoring_Sessions

In [ ]:
```
# this is continous variable and correct data types & there is no null value
```

In [50]:
```
df["Tutoring_Sessions"].unique()
```

Out[50]:
```
array([0, 2, 1, 3, 4, 5, 6, 7, 8], dtype=int64)
```

In [ ]:
```
# Observation: Tutoring_Sessions is a continuous features Wide range of values
 # which are related to Tutoring_Sessions of the student.
```

Family_Income

In [58]:
```
df["Family_Income"].unique()
```

Out[58]:
```
array(['Low', 'Medium', 'High'], dtype=object)
```

In [60]:
```
df["Family_Income"].value_counts()
```

Out[60]:
```
Family_Income
Low       2672
Medium    2666
High      1269
Name: count, dtype: int64
```

In [ ]:
```
# Family_Income is categorical features indicating the Family_Income of the student with
#three categories : 'Low', 'Medium', 'High'.
#among these   low Family_Income  is the most frequent.
```

Teacher_Quality

In [66]:
```
df["Teacher_Quality"].unique()
```

Out[66]:
```
array(['Medium', 'High', 'Low', nan], dtype=object)
```

In [68]:
```
df["Teacher_Quality"].value_counts()
```

Out[68]:
```
Teacher_Quality
Medium    3925
High      1947
Low        657
Name: count, dtype: int64
```

In [70]:
```
df["Teacher_Quality"].isnull().sum()
```

Out[70]:
```
78
```

In [ ]:
```
# Teacher_Quality is categorical features indicating the Teacher_Quality of the student with
#three categories : 'Low', 'Medium', 'High'.
#among these medium Teacher_Quality  is the most frequent.
# this features contain some null value which are indicates the missing value
```

School_Type

In [74]:
```
df["School_Type"].unique()
```

Out[74]:
```
array(['Public', 'Private'], dtype=object)
```

In [76]:
```
df["School_Type"].value_counts()
```

Out[76]:
```
School_Type
Public     4598
Private    2009
Name: count, dtype: int64
```

In [ ]:
```
# Internet_Access is categorical features indicating the Internet_Access of the student with
#two categories : 'Public', 'Private'.
#among these   public school type  is the most frequent.
```

Peer_Influence

In [84]:
```
df["Peer_Influence"].unique()
```

Out[84]:
```
array(['Positive', 'Negative', 'Neutral'], dtype=object)
```

In [86]:
```
df["Peer_Influence"].value_counts()
```

Out[86]:
```
Peer_Influence
Positive    2638
Neutral     2592
Negative    1377
Name: count, dtype: int64
```

In [ ]:
```
# Peer_Influence is categorical features indicating the Peer_Influence of the student with
#three categories : 'Positive', 'Negative', 'Neutral'.
#among these  Positive Peer_Influence  is the most frequent.
```

Physical_Activity

In [92]:
```
df["Physical_Activity"].unique()
```

Out[92]:
```
array([3, 4, 2, 1, 5, 0, 6], dtype=int64)
```

In [ ]:
```
# Observation: Physical_Activity is a continuous features Wide range of values
 # which are related to Physical_Activity of the student.
```

Learning_Disabilities

In [141…
```
df["Learning_Disabilities"].unique()
```

Out[141…
```
array(['No', 'Yes'], dtype=object)
```

In [143…
```
df["Learning_Disabilities"].value_counts()
```

Out[143…
```
Learning_Disabilities
No     5912
Yes     695
Name: count, dtype: int64
```

In [ ]:
```
#Learning_Disabilities is categorical features indicating the Learning_Disabilities of the student with
#two categories : 'No', 'Yes'.
#among these  No Learning_Disabilities is the most frequent.
```

Parental_Education_Level

In [149…
```
df["Parental_Education_Level"].unique()
```

Out[149…
```
array(['High School', 'College', 'Postgraduate', nan], dtype=object)
```

In [151…
```
df["Parental_Education_Level"].value_counts()
```

Out[151…
```
Parental_Education_Level
High School     3223
College         1989
Postgraduate    1305
Name: count, dtype: int64
```

In [58]:
```
df["Parental_Education_Level"].isnull().sum()
```

Out[58]:
```
90
```

In [ ]:
```
# Parental_Education_Levelis categorical features indicating the Parental_Education_Level  of the student with
#three categories : 'High School', 'College', 'Postgraduate'.
#among these  High School  is the most frequent.
# this features contain some null value which are indicates the missing value
```

Distance_from_Home

In [157…
```
df["Distance_from_Home"].unique()
```

Out[157…
```
array(['Near', 'Moderate', 'Far', nan], dtype=object)
```

In [159…
```
df["Distance_from_Home"].value_counts()
```

Out[159…
```
Distance_from_Home
Near        3884
Moderate    1998
Far          658
Name: count, dtype: int64
```

In [161…
```
df["Distance_from_Home"].isnull().sum()
```

Out[161…
```
67
```

In [ ]:
```
# Parental_Education_Levelis categorical features indicating the Parental_Education_Level  of the student with
#three categories : 'Near', 'Moderate', 'Far'.
#among these  Near   is the most frequent.
# this features contain some null value which are indicates the missing value
```

Gender

In [167…
```
df["Gender"].unique()
```

Out[167…
```
array(['Male', 'Female'], dtype=object)
```

In [169…
```
df["Gender"].value_counts()
```

Out[169…
```
Gender
Male      3814
Female    2793
Name: count, dtype: int64
```

In [ ]:
```
# gender is categorical features indicating the gender of the student with two categories :
        #             male and female.
#among these male is the most frequent.
```

Exam_Score

In [60]:
```
df["Exam_Score"].unique()
```

Out[60]:
```
array([ 67,  61,  74,  71,  70,  66,  69,  72,  68,  65,  64,  60,  63,
        62, 100,  76,  79,  73,  78,  89,  75,  59,  86,  97,  83,  84,
        80,  58,  94,  55,  92,  82,  77, 101,  88,  91,  99,  87,  57,
        96,  98,  95,  85,  93,  56], dtype=int64)
```

In [ ]:
```
# Observation: Exam_Score is a continuous features Wide range of values
 # which are related to Tutoring_Sessions of the student.
```

In [68]:
```
discrete=['Attendance','Parental_Involvement','Access_to_Resources','Extracurricular_Activities',
          'Motivation_Level','Internet_Access','Family_Income','Teacher_Quality','School_Type',
          'Peer_Influence','Learning_Disabilities','Parental_Education_Level','Gender']
discrete
```

Out[68]:
```
['Attendance',
 'Parental_Involvement',
 'Access_to_Resources',
 'Extracurricular_Activities',
 'Motivation_Level',
 'Internet_Access',
 'Family_Income',
 'Teacher_Quality',
 'School_Type',
 'Peer_Influence',
 'Learning_Disabilities',
 'Parental_Education_Level',
 'Gender']
```

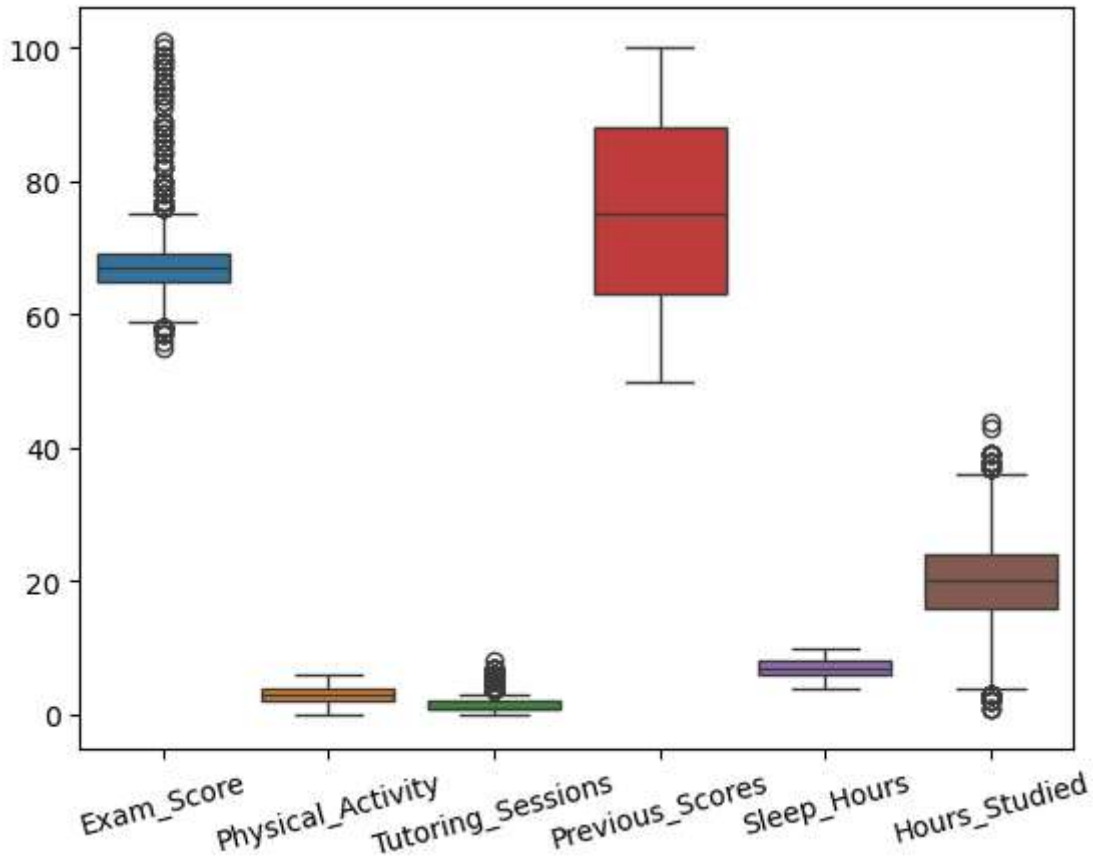
In [ ]:
```

```

In [78]:
```
continuous=['Exam_Score','Distance_from_Home','Physical_Activity','Tutoring_Sessions','Previous_Scores'
          ,'Sleep_Hours','Hours_Studied']
```

In [80]:
```
continuous
```

Out[80]:
```
['Exam_Score',
 'Distance_from_Home',
 'Physical_Activity',
 'Tutoring_Sessions',
 'Previous_Scores',
 'Sleep_Hours',
 'Hours_Studied']
```

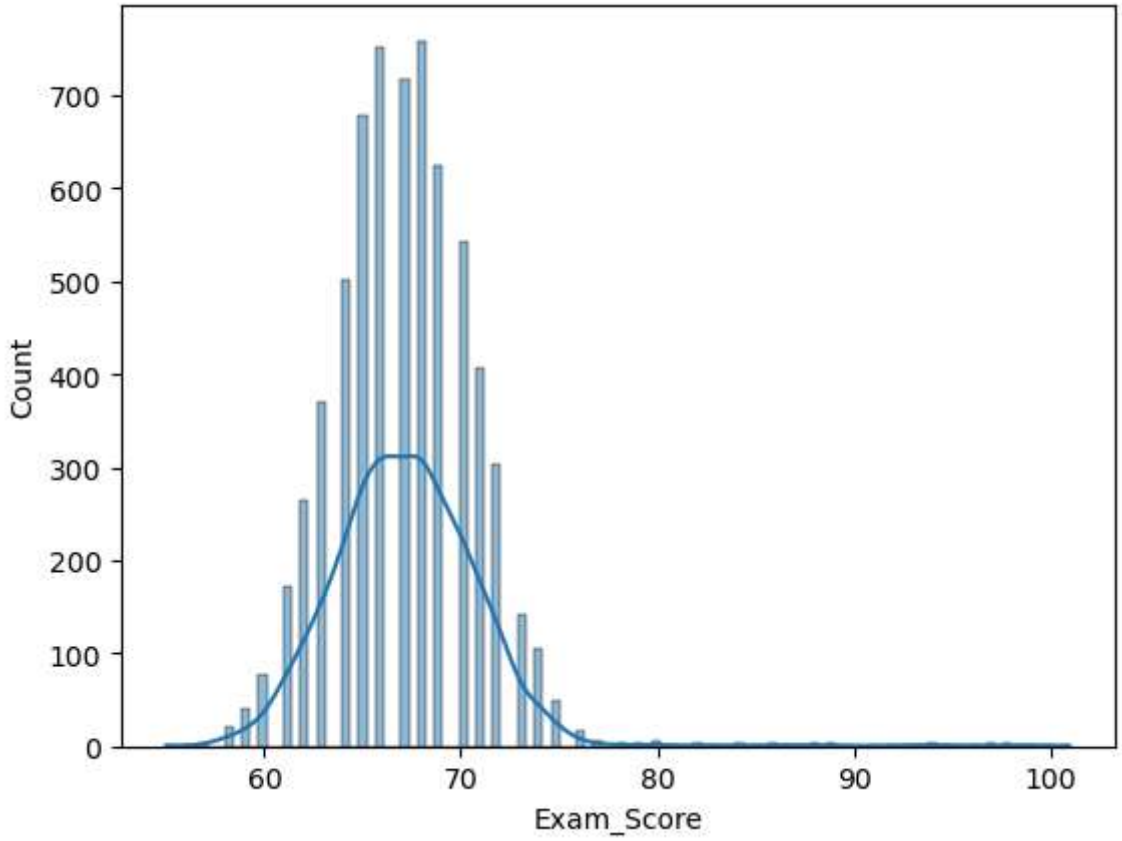
In [86]:
```
sns.boxplot(df[continuous])
plt.xticks(rotation=15)
plt.show()
```



In [ ]:
```
# observe:- exam score ,tutoring sessions,hours studied have some outliers.
```

In [102…
```
sns.histplot(df["Exam_Score"],kde=True)
```

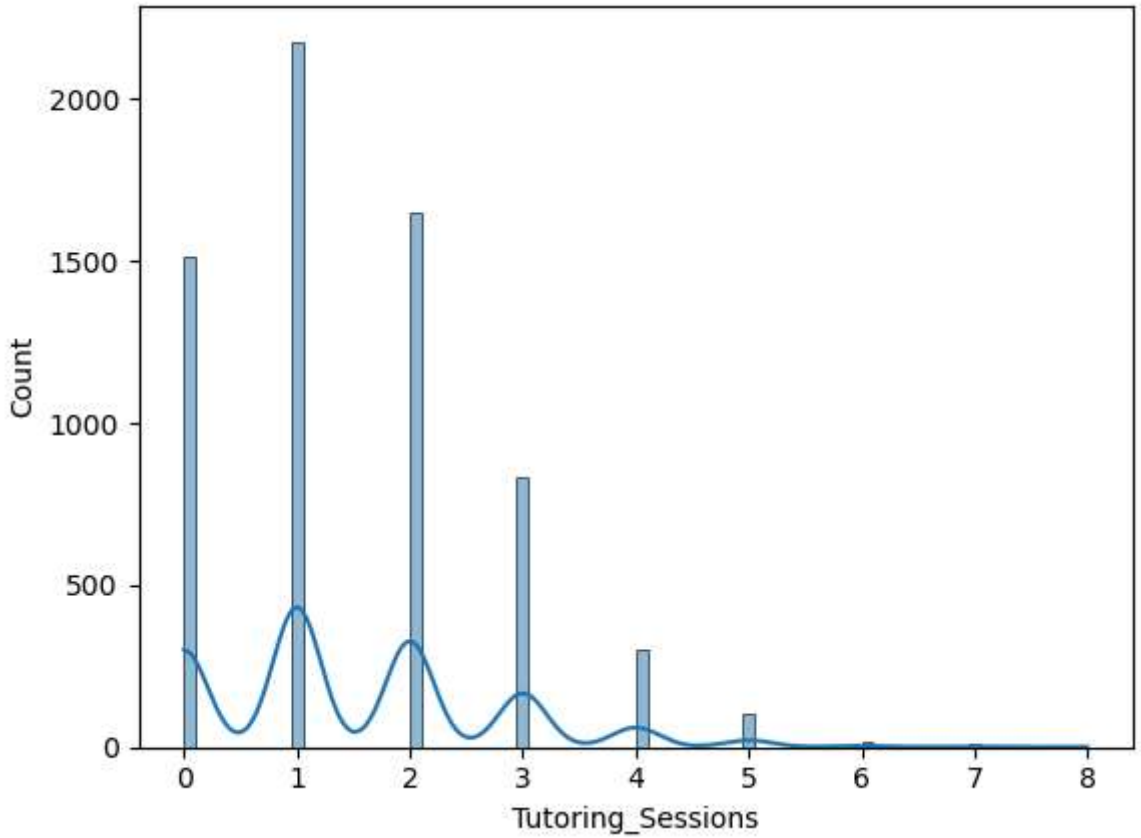
Out[102…
```
<Axes: xlabel='Exam_Score', ylabel='Count'>
```



In [ ]:
```
# observe :- here we observe right side skewness available  in this features
```

In [104…
```
sns.histplot(df["Tutoring_Sessions"],kde=True)
```

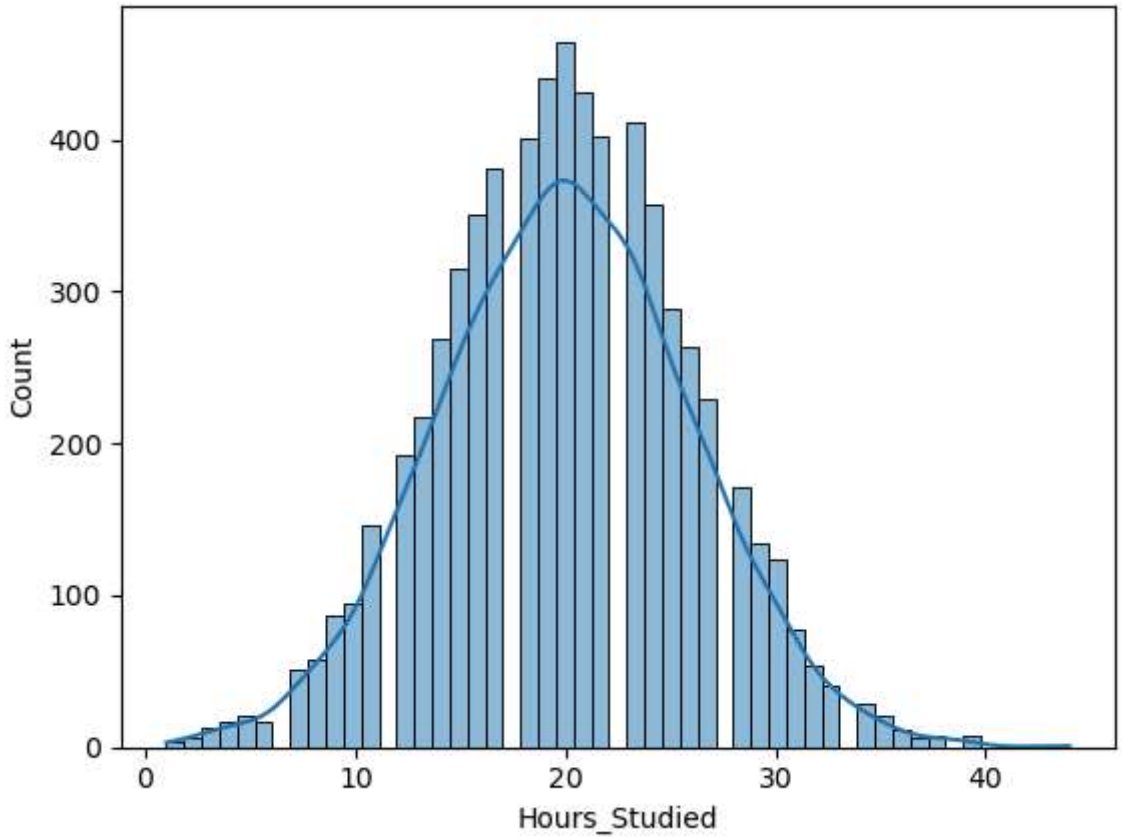
Out[104…
```
<Axes: xlabel='Tutoring_Sessions', ylabel='Count'>
```



In [ ]:
```
# observe :- here we observe right side skewness available  in this features
```

In [106…
```
sns.histplot(df["Hours_Studied"],kde=True)
```

Out[106…
```
<Axes: xlabel='Hours_Studied', ylabel='Count'>
```



In [ ]:
```
# observe :- here we observe right side skewness available  in this features
```

step 3 Data Preprocessing

In [ ]:
```
# Data cleaning
# demension reduction
# Data transformation
```

In [189…
```
# in this columns some missing value
missing_value=["Teacher_Quality","Parental_Education_Level","Distance_from_Home"]
```

In [29]:
```
# fill the missing value with mode
df["Teacher_Quality"]=df["Teacher_Quality"].fillna(df["Teacher_Quality"].mode()[0])
df["Parental_Education_Level"]=df["Parental_Education_Level"].fillna(df["Parental_Education_Level"].mode()[0])
df["Distance_from_Home"]=df["Distance_from_Home"].fillna(df["Distance_from_Home"].mode()[0])
```

In [ ]:
```
# Reason:-
#Filling missing values with the mode (the most frequent value in a column)
```

```
#is a common data preprocessing technique, especially for categorical variables.
```

In [ ]:
```
# Data cleaning
# treat the outliers
```

In [ ]:
```
#Outliers in exam score ,tutoring sessions,hours studied represent real and clinically.
#which are critical risk factors for  student performance.
#Removing  these values might hide important patterns so that retain as it.
```

step 4 & 5 Data analysis & visualization

In [33]:
```
df.head(2)
```

Out[33]:

	Studied_Hours	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Scores	Motivation_I
0	23	84	Low	High	No	7	73	
1	19	64	Low	Medium	No	8	59	

In [35]:
```
df.describe()
```

Out[35]:

	Studied_Hours	Attendance	Sleep_Hours	Previous_Scores	Tutoring_Sessions	Physical_Activity	Exam_Score
count	6607.000000	6607.000000	6607.00000	6607.000000	6607.000000	6607.000000	6607.000000
mean	19.975329	79.977448	7.02906	75.070531	1.493719	2.967610	67.235659
std	5.990594	11.547475	1.46812	14.399784	1.230570	1.031231	3.890456
min	1.000000	60.000000	4.00000	50.000000	0.000000	0.000000	55.000000
25%	16.000000	70.000000	6.00000	63.000000	1.000000	2.000000	65.000000
50%	20.000000	80.000000	7.00000	75.000000	1.000000	3.000000	67.000000
75%	24.000000	90.000000	8.00000	88.000000	2.000000	4.000000	69.000000
max	44.000000	100.000000	10.00000	100.000000	8.000000	6.000000	101.000000

In [39]:
```
df.describe(include="object").T
```

Out[39]:

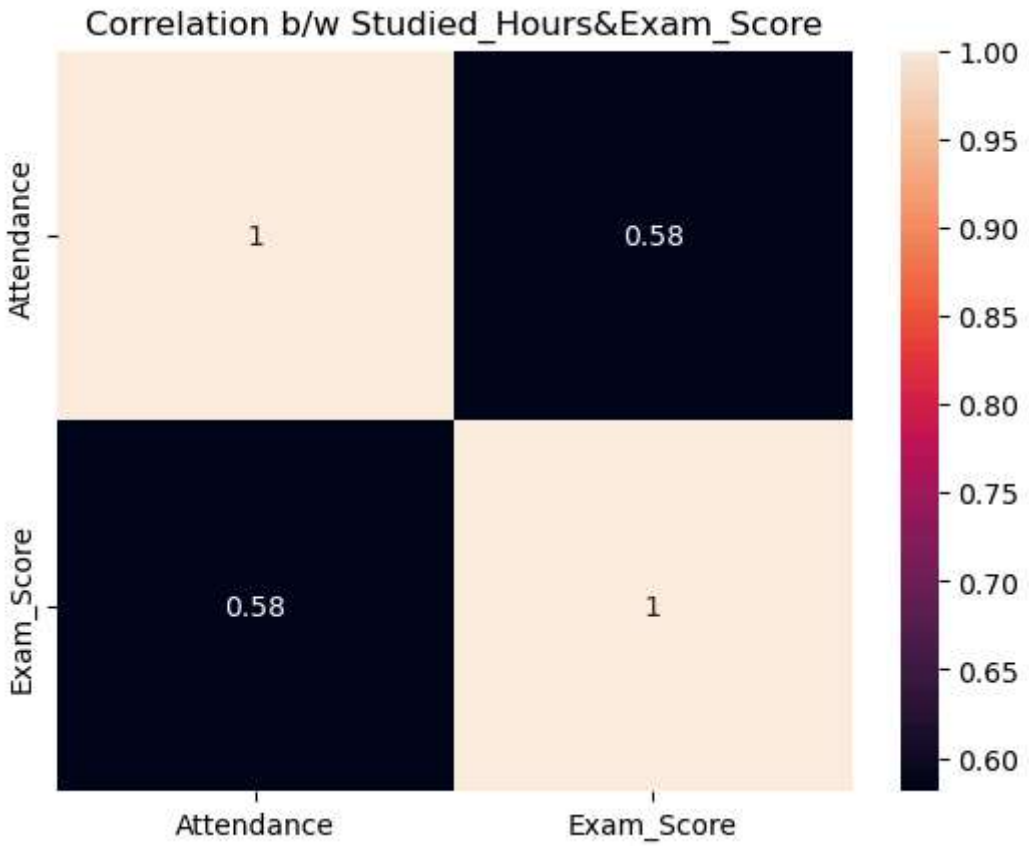
	count	unique	top	freq
Parental_Involvement	6607	3	Medium	3362
Access_to_Resources	6607	3	Medium	3319
Extracurricular_Activities	6607	2	Yes	3938
Motivation_Level	6607	3	Medium	3351
Internet_Access	6607	2	Yes	6108
Family_Income	6607	3	Low	2672
Teacher_Quality	6607	3	Medium	4003
School_Type	6607	2	Public	4598
Peer_Influence	6607	3	Positive	2638
Learning_Disabilities	6607	2	No	5912
Parental_Education_Level	6607	3	High School	3313
Distance_from_Home	6607	3	Near	3951
Gender	6607	2	Male	3814

In [21]:
```
score=df[["Studied_Hours","Exam_Score"]].corr()
score
```

Out[21]:

	Studied_Hours	Exam_Score
Studied_Hours	1.000000	0.445455
Exam_Score	0.445455	1.000000

In [77]:
```
sns.heatmap(score ,annot=True)
plt.title("Correlation b/w Studied_Hours&Exam_Score")
plt.savefig("Studied_HoursExam_Score.jpg")
plt.show()
```



In [ ]:
```
# the correlation b/w studied_ hours and Exam_score is weak correlation
# so that there is some effect on exam_score  due to the studied_hours.
```

In [ ]:
```
```

In [215…
```
df[["Attendance","Exam_Score"]].corr()
```

Out[215…

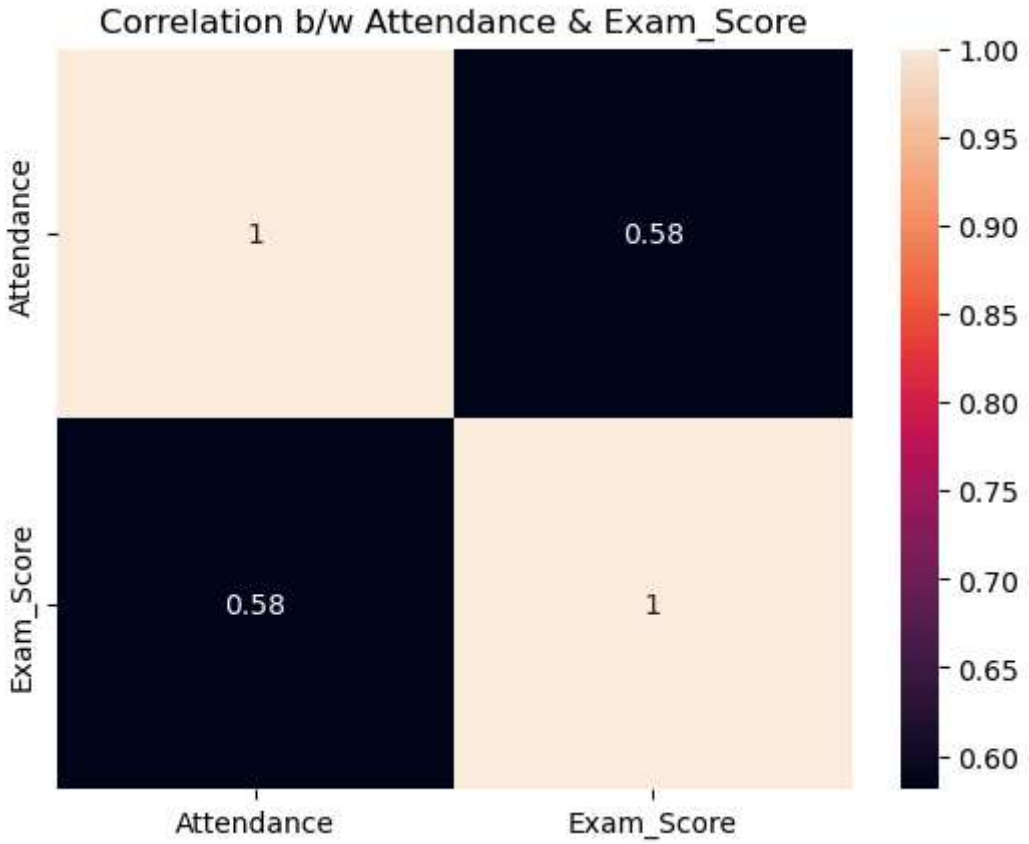
	Attendance	Exam_Score
Attendance	1.000000	0.581072
Exam_Score	0.581072	1.000000

In [35]:
```
score= df[["Attendance","Exam_Score"]].corr()
score
```

Out[35]:

	Attendance	Exam_Score
Attendance	1.000000	0.581072
Exam_Score	0.581072	1.000000

In [75]:
```
sns.heatmap(score ,annot=True)
plt.title("Correlation b/w Attendance & Exam_Score")
plt.savefig("AttendanceExam_Score.jpg")
plt.show()
```



In [ ]:
```
# the correlation b/w attendance and Exam_score is moderate correlation
# so that there is few effect on exam_score  due to the attendance.
```

In [ ]:
```

```

In [131…
```
df.groupby("Parental_Involvement")["Exam_Score"].min()
```

Out[131…
```
Parental_Involvement
High      57
Low       56
Medium    55
Name: Exam_Score, dtype: int64
```

In [133…
```
df.groupby("Parental_Involvement")["Exam_Score"].max()
```

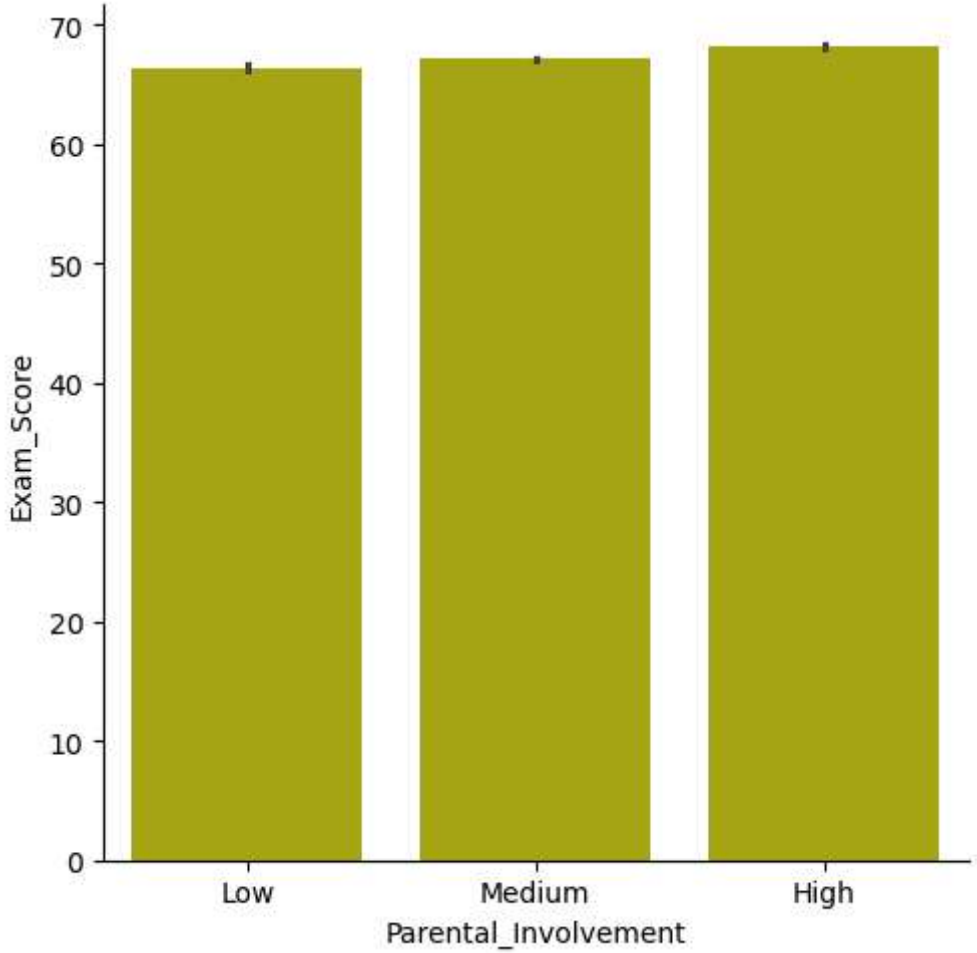
Out[133…
```
Parental_Involvement
High      100
Low       101
Medium     97
Name: Exam_Score, dtype: int64
```

In [135…
```
df.groupby("Parental_Involvement")["Exam_Score"].mean()
```

Out[135…
```
Parental_Involvement
High      68.092767
Low       66.358265
Medium    67.098156
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Parental_Involvement
# so that there is no effect on exam_score  due to the Parental_Involvement.
```

In [11]:
```
sns.catplot(x="Parental_Involvement",y="Exam_Score",data=df,kind="bar",color="y")
plt.show()
plt.savefig("Parental_Involvement.jpg")
```



<Figure size 640x480 with 0 Axes>

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Access_to_Resources
# so that there is no effect on exam_score  due to the Access_to_Resources.
```

In [137…
```
df.groupby("Access_to_Resources")["Exam_Score"].mean()
```

Out[137…
```
Access_to_Resources
High      68.092152
Low       66.203351
Medium    67.134378
Name: Exam_Score, dtype: float64
```

In [141…
```
df.groupby("Access_to_Resources")["Exam_Score"].min()
```

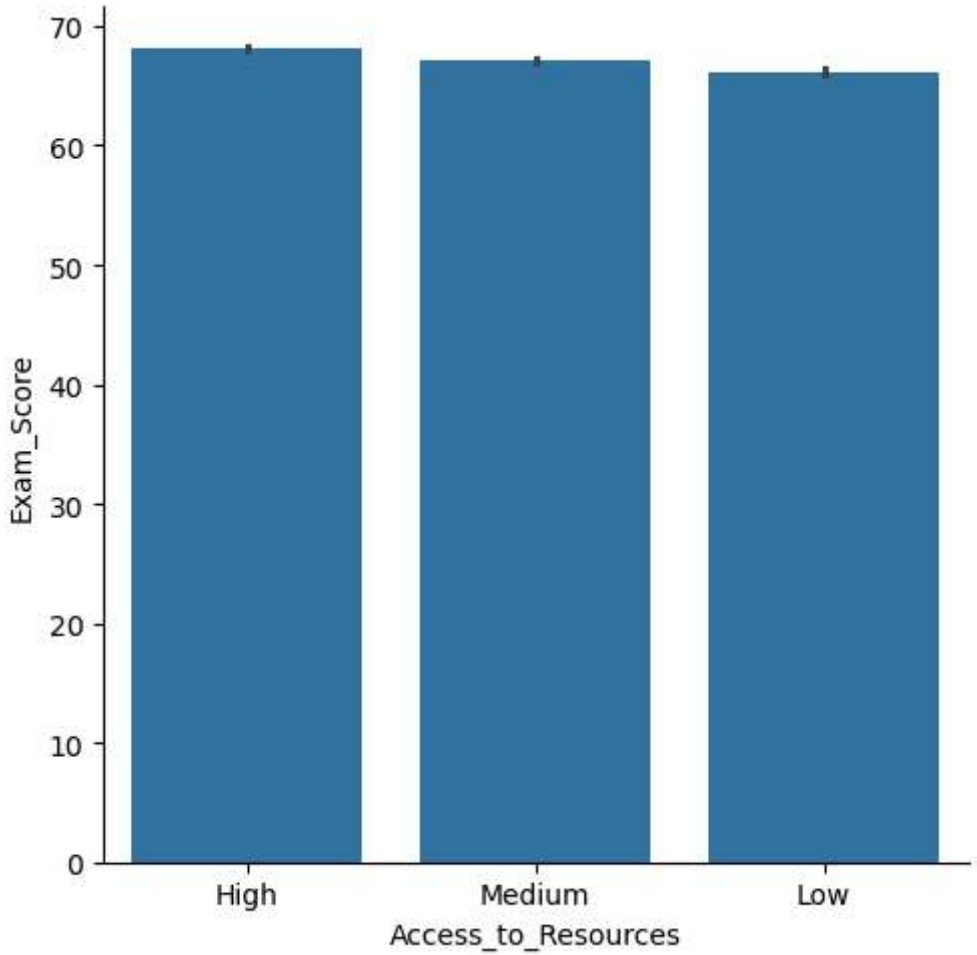
Out[141…
```
Access_to_Resources
High      56
Low       55
Medium    58
Name: Exam_Score, dtype: int64
```

In [139…
```
df.groupby("Access_to_Resources")["Exam_Score"].max()
```

Out[139…
```
Access_to_Resources
High       99
Low        98
Medium    101
Name: Exam_Score, dtype: int64
```

In [ ]:
```
```

In [17]:
```
sns.catplot(x="Access_to_Resources",y="Exam_Score",data=df,kind="bar")
plt.savefig("Access_to_Resources.jpg")
plt.show()
```



In [ ]:

In [173…
```
df.groupby("Extracurricular_Activities")["Exam_Score"].min()
```

Out[173…
```
Extracurricular_Activities
No     55
Yes    57
Name: Exam_Score, dtype: int64
```

In [177…
```
df.groupby("Extracurricular_Activities")["Exam_Score"].max()
```

Out[177…
```
Extracurricular_Activities
No      97
Yes    101
Name: Exam_Score, dtype: int64
```

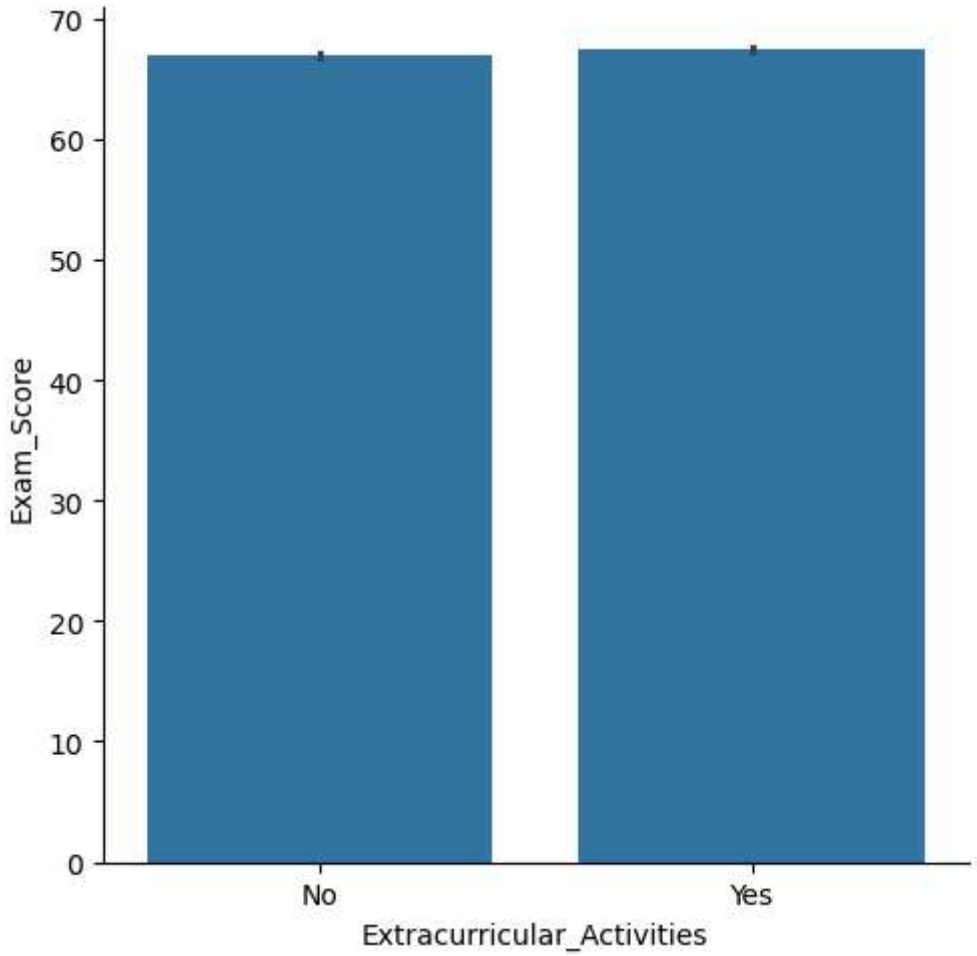
In [175…
```
df.groupby("Extracurricular_Activities")["Exam_Score"].mean()
```

Out[175…
```
Extracurricular_Activities
No     66.931435
Yes    67.441849
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Extracurricular_Activities
# so that there is no effect on exam_score  due to the Extracurricular_Activities.
```

In [183…
```
sns.catplot(x="Extracurricular_Activities",y="Exam_Score",data=df,kind="bar")
```

Out[183…
```
<seaborn.axisgrid.FacetGrid at 0x28ccaa54620>
```



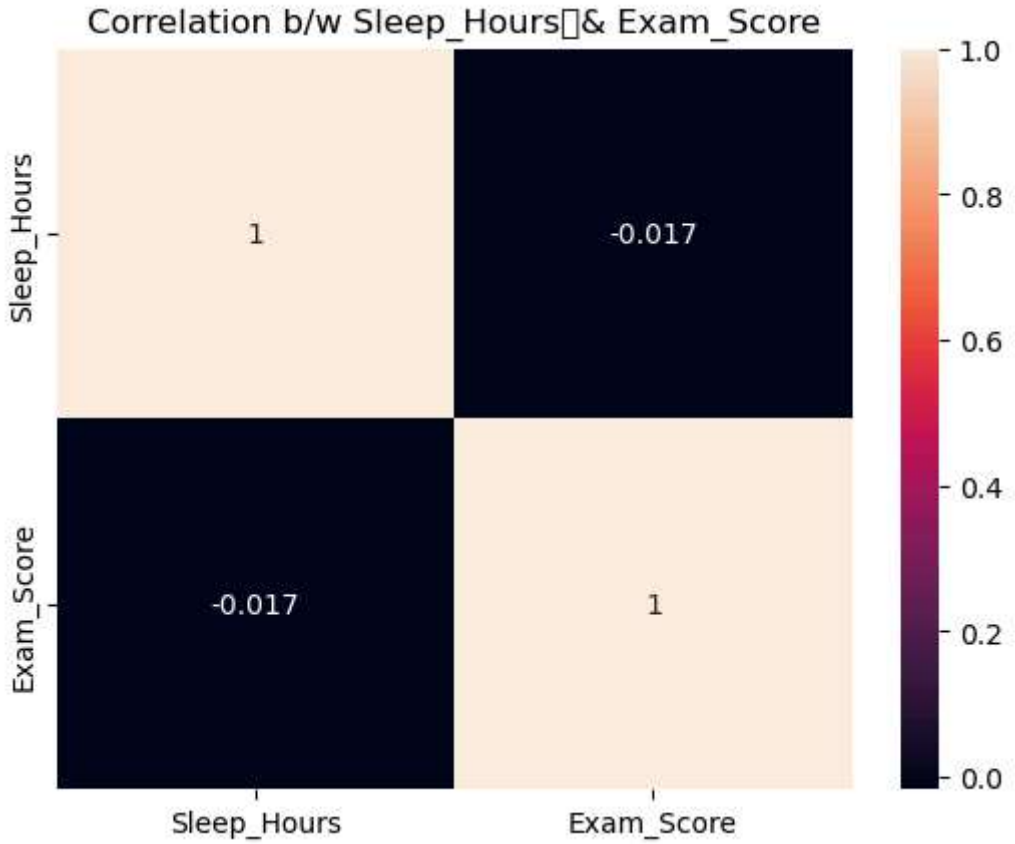
In [ ]:

In [43]:
```
corr=df[["Sleep_Hours","Exam_Score"]].corr()
corr
```

Out[43]:

	Sleep_Hours	Exam_Score
Sleep_Hours	1.000000	-0.017022
Exam_Score	-0.017022	1.000000

In [73]:
```
sns.heatmap(corr,annot=True)
plt.title("Correlation b/w Sleep_Hours  & Exam_Score")
plt.savefig("Sleep_HourExam_Score")
plt.show()
```



In [ ]:
```
# the correlation b/w Sleep_Hours and Exam_score is (-ve) correlation
# so that there is few effect on exam_score  due to the sleep_hours.
# if student sleep_hours is more atumatically thier exam score is less  that s means vice versa
```

In [ ]:
```

```

In [219…
```
corr=df[["Previous_Scores","Exam_Score"]].corr()
corr
```

Out[219…

	Previous_Scores	Exam_Score
Previous_Scores	1.000000	0.175079
Exam_Score	0.175079	1.000000

In [221…
```
sns.heatmap(corr,annot=True)
```

Out[221… <Axes: >

In [ ]:
```
# the correlation b/w previous_Scores and Exam_score is weak correlation
# so that there is few effect on exam_score  due to the previous_Scores.
```

In [ ]:
```

```

In [223…
```
df.groupby("Motivation_Level")["Exam_Score"].max()
```

Out[223…
```
Motivation_Level
High       98
Low       101
Medium    100
Name: Exam_Score, dtype: int64
```

In [225…
```
df.groupby("Motivation_Level")["Exam_Score"].min()
```

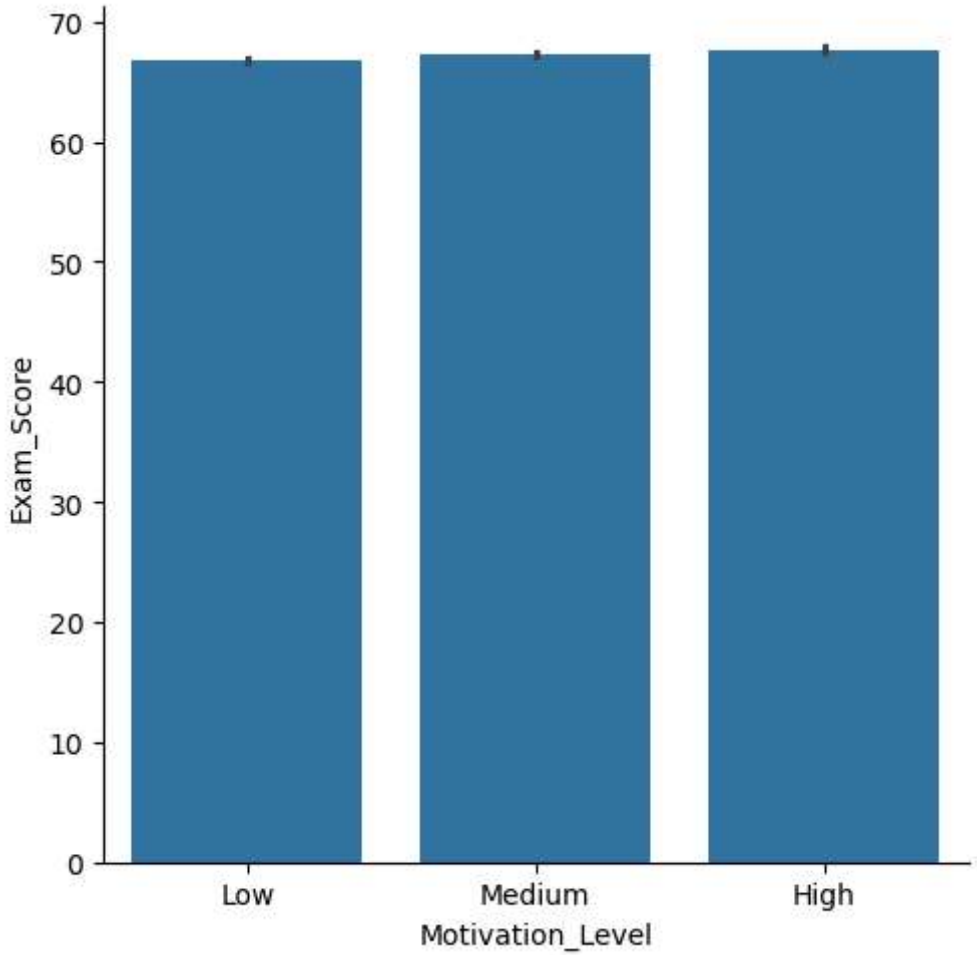
Out[225…
```
Motivation_Level
High      57
Low       57
Medium    55
Name: Exam_Score, dtype: int64
```

In [227…
```
df.groupby("Motivation_Level")["Exam_Score"].mean()
```

Out[227…
```
Motivation_Level
High      67.704321
Low       66.752194
Medium    67.330648
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Motivation_Level
# so that there is no effect on exam_score  due to the Motivation_Level.
```

In [231…
```
sns.catplot(x="Motivation_Level",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [ ]:

In [233…
```
df.groupby("Internet_Access")["Exam_Score"].max()
```

Out[233…
```
Internet_Access
No     101
Yes    100
Name: Exam_Score, dtype: int64
```

In [235…
```
df.groupby("Internet_Access")["Exam_Score"].min()
```

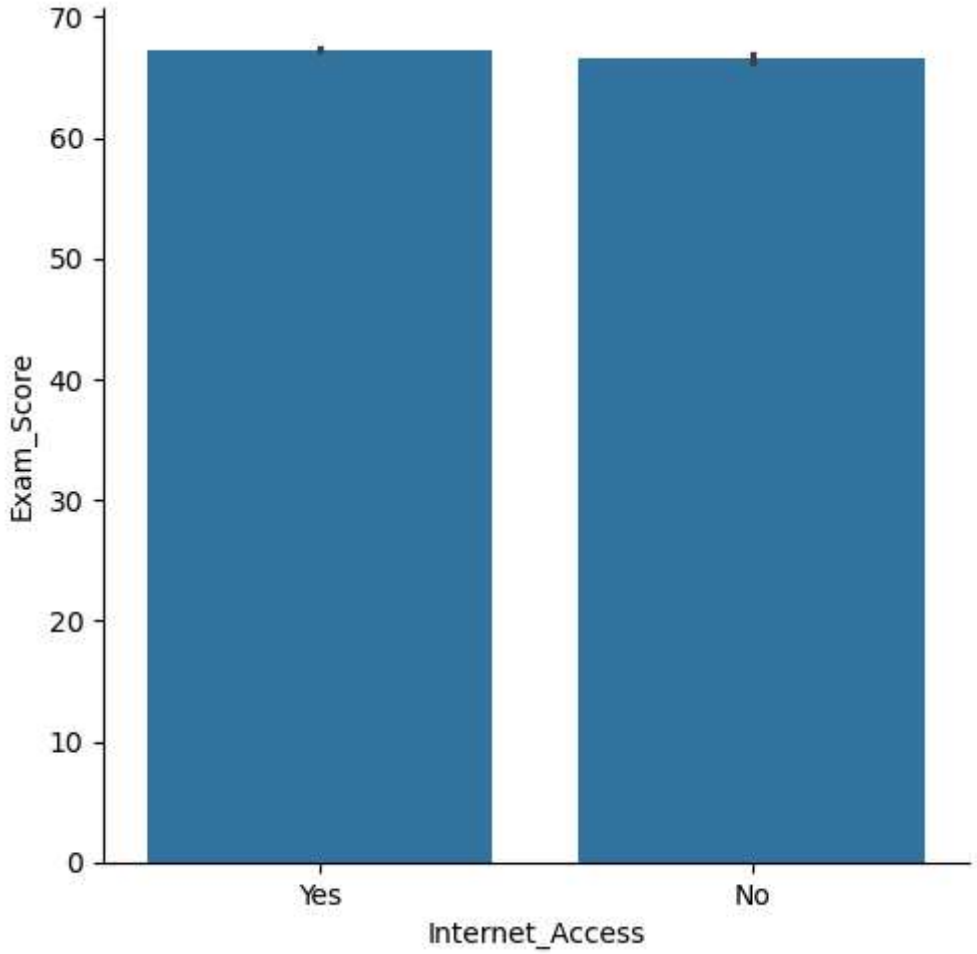
Out[235…
```
Internet_Access
No     58
Yes    55
Name: Exam_Score, dtype: int64
```

In [237…
```
df.groupby("Internet_Access")["Exam_Score"].mean()
```

Out[237…
```
Internet_Access
No     66.535070
Yes    67.292895
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Internet_Access
# so that there is no effect on exam_score  due to the Internet_Access.
```

In [239…
```
sns.catplot(x="Internet_Access",y="Exam_Score",data=df,kind="bar")
plt.show()
```



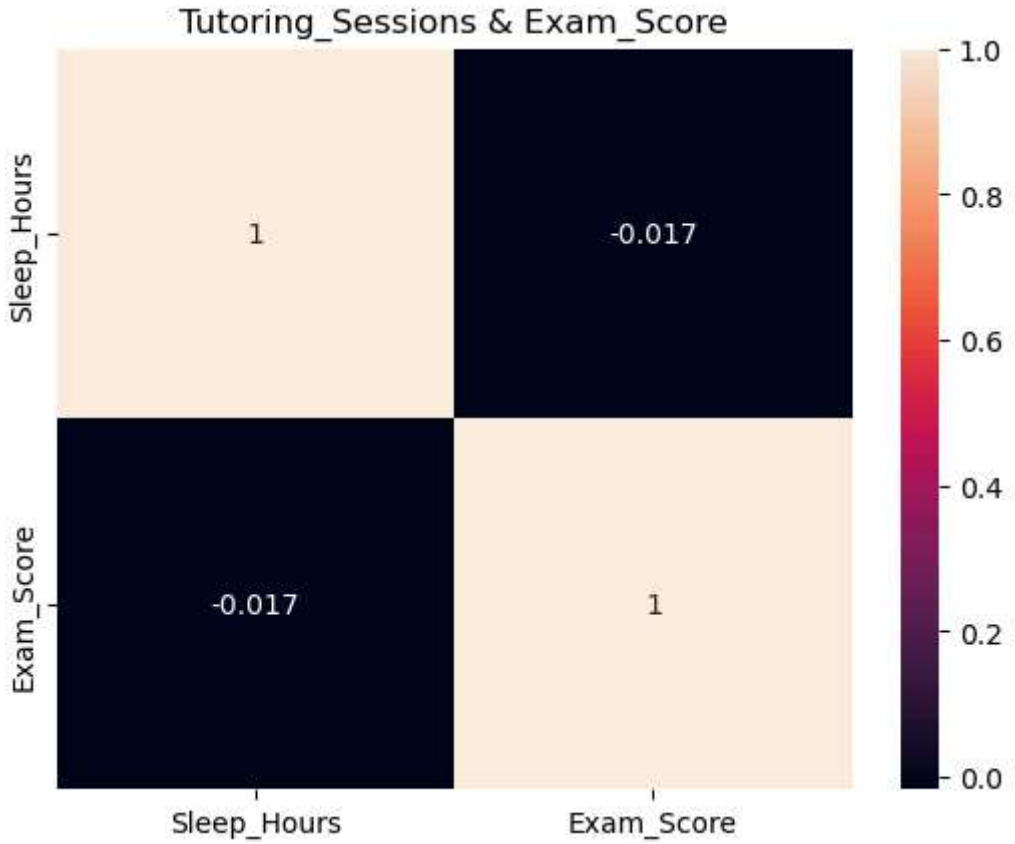
In [ ]:
```
```

In [245…
```
corr=df[["Tutoring_Sessions","Exam_Score"]].corr()
corr
```

Out[245…

	Tutoring_Sessions	Exam_Score
Tutoring_Sessions	1.000000	0.156525
Exam_Score	0.156525	1.000000

In [71]:
```
sns.heatmap(corr,annot=True)
plt.title("Tutoring_Sessions & Exam_Score")
plt.savefig("Tutoring_SessionsExam_Score")
plt.show()
```



In [ ]:
```
# the correlation b/w Tutoring_Sessions and Exam_score is weak correlation
# so that there is few effect on exam_score  due to the Tutoring_Sessions.
```

In [ ]:
```

```

In [251…
```
df.groupby("Family_Income")["Exam_Score"].max()
```

Out[251…
```
Family_Income
High      101
Low        97
Medium     99
Name: Exam_Score, dtype: int64
```

In [275…
```
df.groupby("Family_Income")["Exam_Score"].min()
```

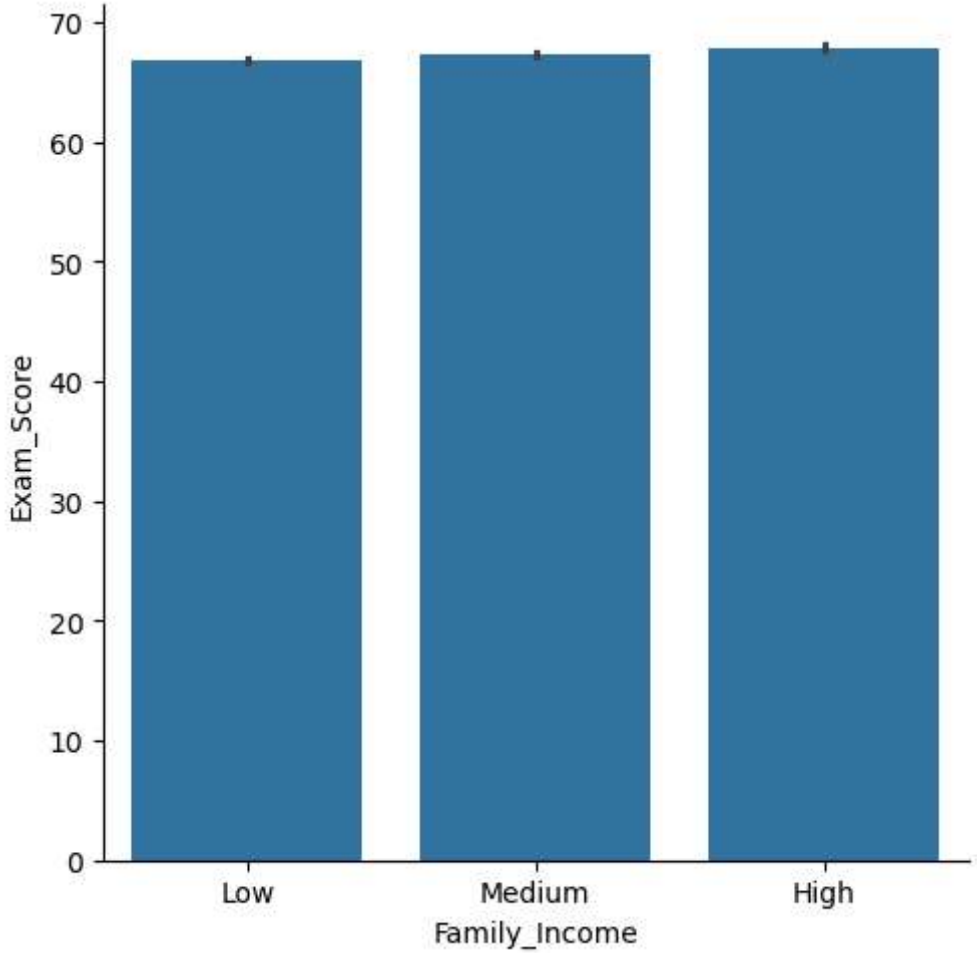
Out[275…
```
Family_Income
High      58
Low       55
Medium    57
Name: Exam_Score, dtype: int64
```

In [277…
```
df.groupby("Family_Income")["Exam_Score"].mean()
```

Out[277…
```
Family_Income
High      67.842396
Low       66.848428
Medium    67.334959
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Family_Income
# so that there is no effect on exam_score  due to the Family_Income.
```

In [279…
```
sns.catplot(x="Family_Income",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [ ]:

In [259…
```
df.groupby("Teacher_Quality")["Exam_Score"].max()
```

Out[259…
```
Teacher_Quality
High      101
Low        94
Medium    100
Name: Exam_Score, dtype: int64
```

In [281…
```
df.groupby("Teacher_Quality")["Exam_Score"].min()
```

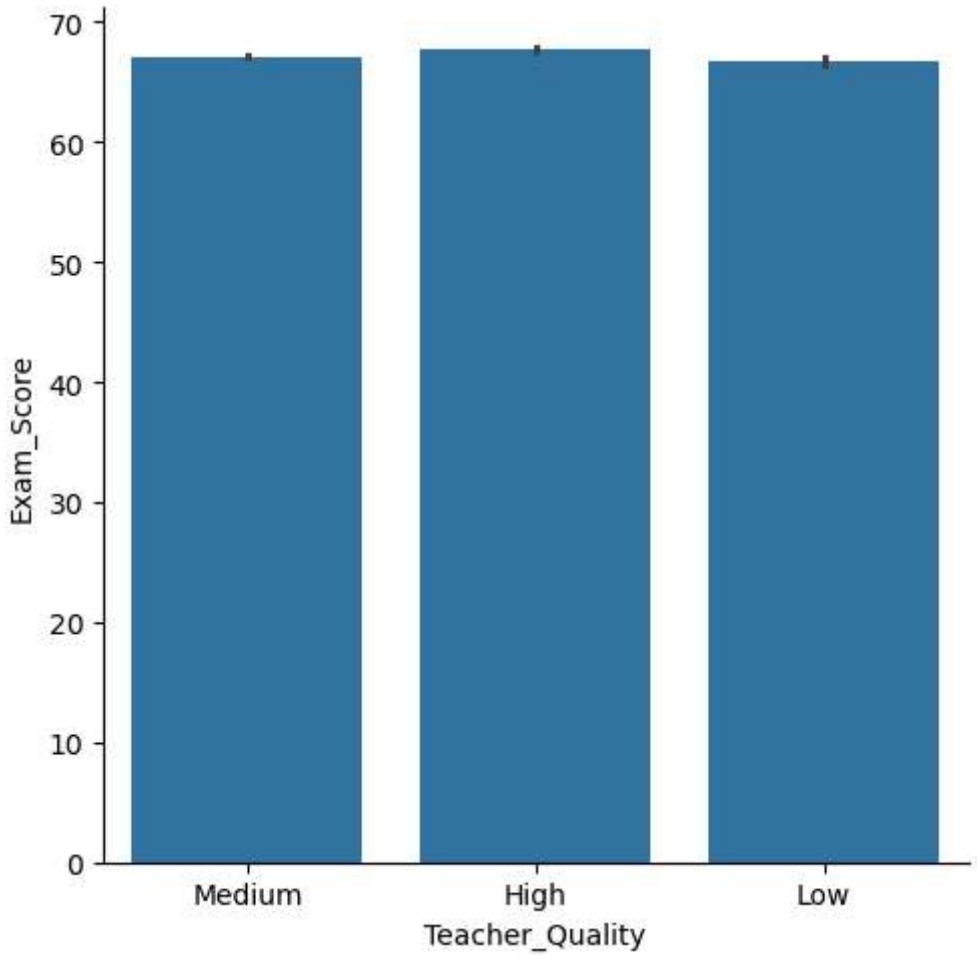
Out[281…
```
Teacher_Quality
High      58
Low       58
Medium    55
Name: Exam_Score, dtype: int64
```

In [285…
```
df.groupby("Teacher_Quality")["Exam_Score"].mean()
```

Out[285…
```
Teacher_Quality
High      67.676939
Low       66.753425
Medium    67.109299
Name: Exam_Score, dtype: float64
```

In [265…
```
# the mean,minmum,maximum exam_score is not effected by Teacher_Quality
# so that there is no effect on exam_score  due to the Teacher_Quality
```

In [287…
```
sns.catplot(x="Teacher_Quality",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [ ]:

In [269…
```
df.groupby("School_Type")["Exam_Score"].max()
```

Out[269…
```
School_Type
Private    100
Public     101
Name: Exam_Score, dtype: int64
```

In [289…
```
df.groupby("School_Type")["Exam_Score"].min()
```

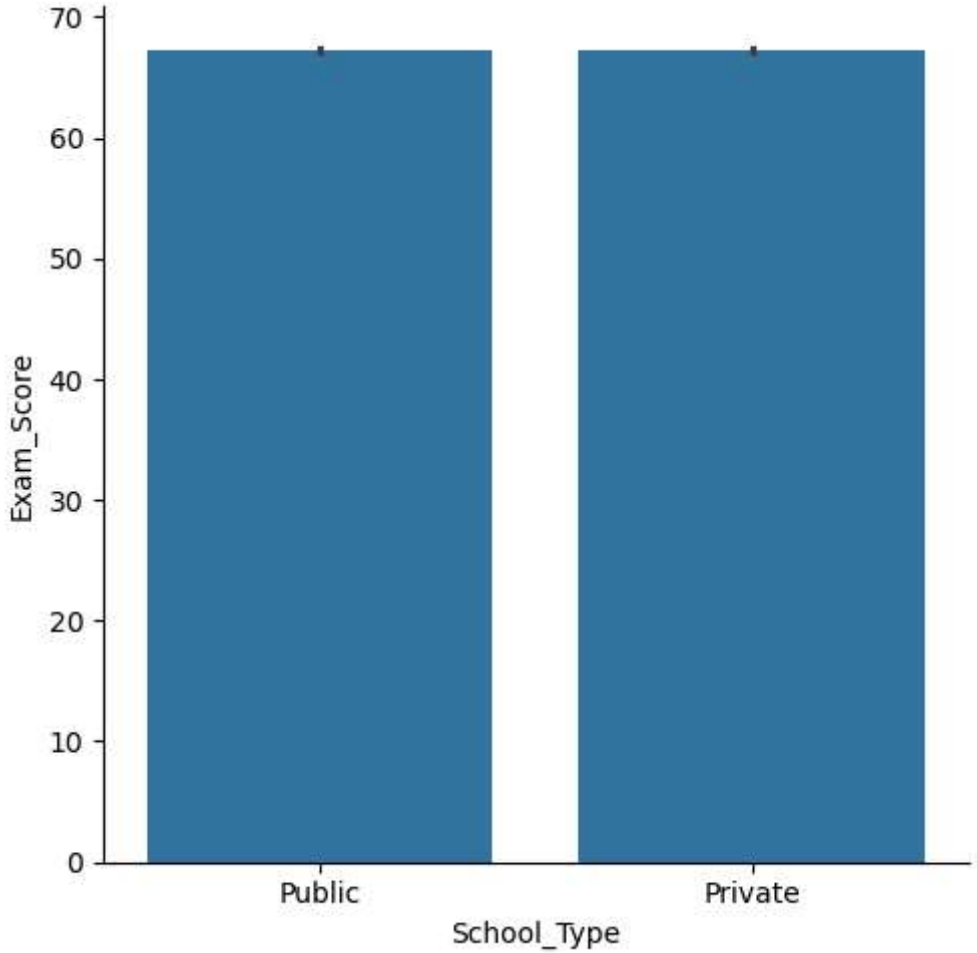
Out[289…
```
School_Type
Private    56
Public     55
Name: Exam_Score, dtype: int64
```

In [291…
```
df.groupby("School_Type")["Exam_Score"].mean()
```

Out[291…
```
School_Type
Private    67.287705
Public     67.212919
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by School_Type
# so that there is no effect on exam_score  due to the School_Type.
```

In [293…
```
sns.catplot(x="School_Type",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [ ]:

In [295…
```
df.groupby("Peer_Influence")["Exam_Score"].max()
```

Out[295…
```
Peer_Influence
Negative     99
Neutral      97
Positive    101
Name: Exam_Score, dtype: int64
```

In [297…
```
df.groupby("Peer_Influence")["Exam_Score"].min()
```

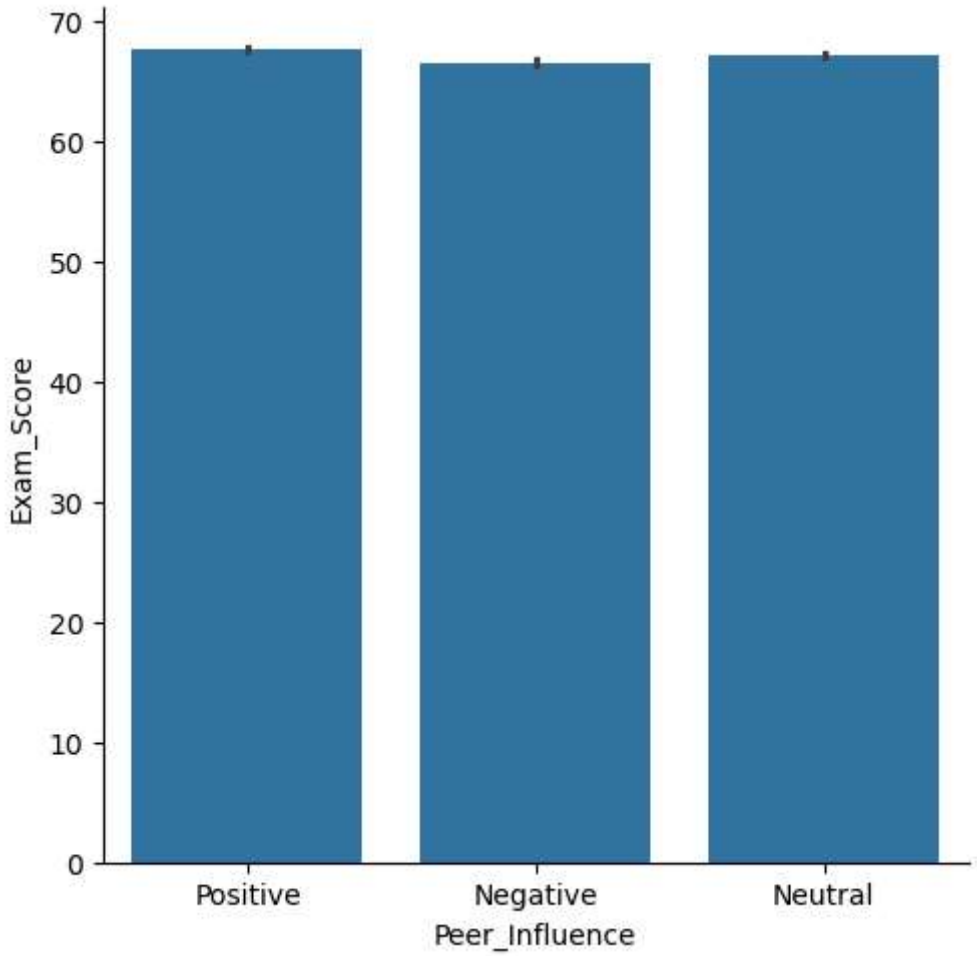
Out[297…
```
Peer_Influence
Negative    55
Neutral     57
Positive    57
Name: Exam_Score, dtype: int64
```

In [299…
```
df.groupby("Peer_Influence")["Exam_Score"].mean()
```

Out[299…
```
Peer_Influence
Negative    66.564270
Neutral     67.197917
Positive    67.623199
Name: Exam_Score, dtype: float64
```

In [301…
```
# the mean,minmum,maximum exam_score is not effected by Peer_Influence
# so that there is no effect on exam_score  due to the Peer_Influence
```

In [303…
```
sns.catplot(x="Peer_Influence",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [309…
```
corr=df[["Physical_Activity","Exam_Score"]].corr()
corr
```

Out[309…

	Physical_Activity	Exam_Score
Physical_Activity	1.000000	0.027824
Exam_Score	0.027824	1.000000

In [311…
```
sns.heatmap(corr,annot=True)
```

Out[311… <Axes: >

In [ ]:
```
# the correlation b/w Physical_Activity and Exam_score is weak correlation
# so that there is few effect on exam_score  due to the Physical_Activity.
```

In [ ]:
```

```

In [313…
```
df.groupby("Learning_Disabilities")["Exam_Score"].max()
```

Out[313…
```
Learning_Disabilities
No     101
Yes     89
Name: Exam_Score, dtype: int64
```

In [315…
```
df.groupby("Learning_Disabilities")["Exam_Score"].min()
```

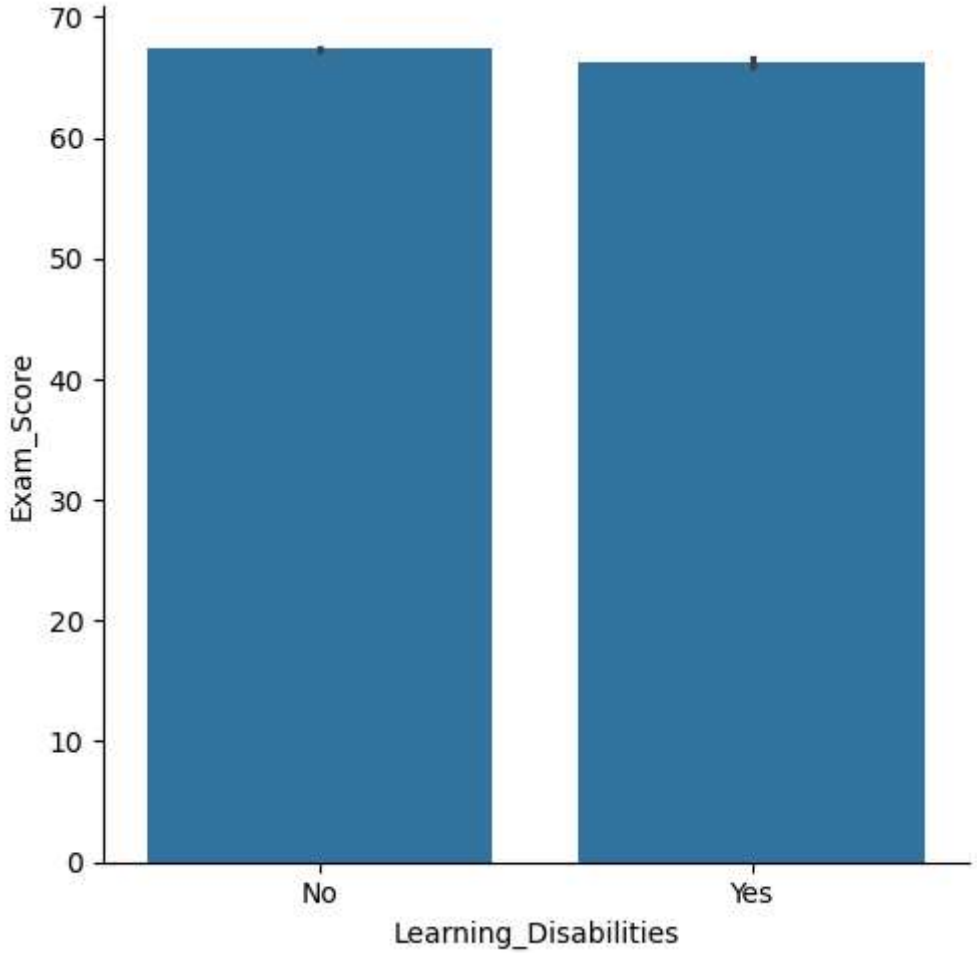
Out[315…
```
Learning_Disabilities
No     55
Yes    57
Name: Exam_Score, dtype: int64
```

In [317…
```
df.groupby("Learning_Disabilities")["Exam_Score"].mean()
```

Out[317…
```
Learning_Disabilities
No     67.349120
Yes    66.270504
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Learning_Disabilities
# so that there is no effect on exam_score  due to the Learning_Disabilities
```

In [319…
```
sns.catplot(x="Learning_Disabilities",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [ ]:

In [321…
```
df.groupby("Parental_Education_Level")["Exam_Score"].max()
```

Out[321…
```
Parental_Education_Level
College          100
High School      101
Postgraduate      98
Name: Exam_Score, dtype: int64
```

In [323…
```
df.groupby("Parental_Education_Level")["Exam_Score"].min()
```

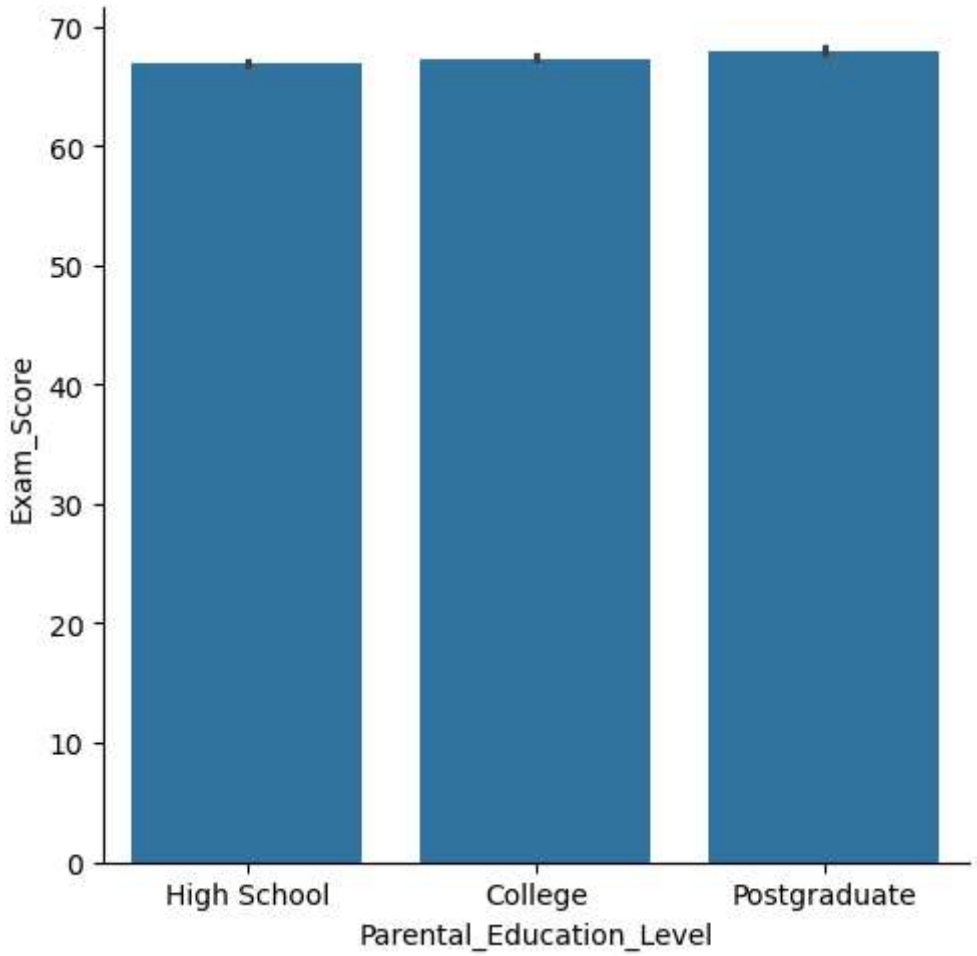
Out[323…
```
Parental_Education_Level
College          56
High School      55
Postgraduate     57
Name: Exam_Score, dtype: int64
```

In [325…
```
df.groupby("Parental_Education_Level")["Exam_Score"].mean()
```

Out[325…
```
Parental_Education_Level
College          67.315737
High School      66.893577
Postgraduate     67.970881
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Parental_Education_Level
# so that there is no effect on exam_score  due to the Parental_Education_Level
```

In [327…
```
sns.catplot(x="Parental_Education_Level",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [ ]:

In [329…
```
df.groupby("Distance_from_Home")["Exam_Score"].min()
```

Out[329…
```
Distance_from_Home
Far         56
Moderate    57
Near        55
Name: Exam_Score, dtype: int64
```

In [331…
```
df.groupby("Distance_from_Home")["Exam_Score"].max()
```

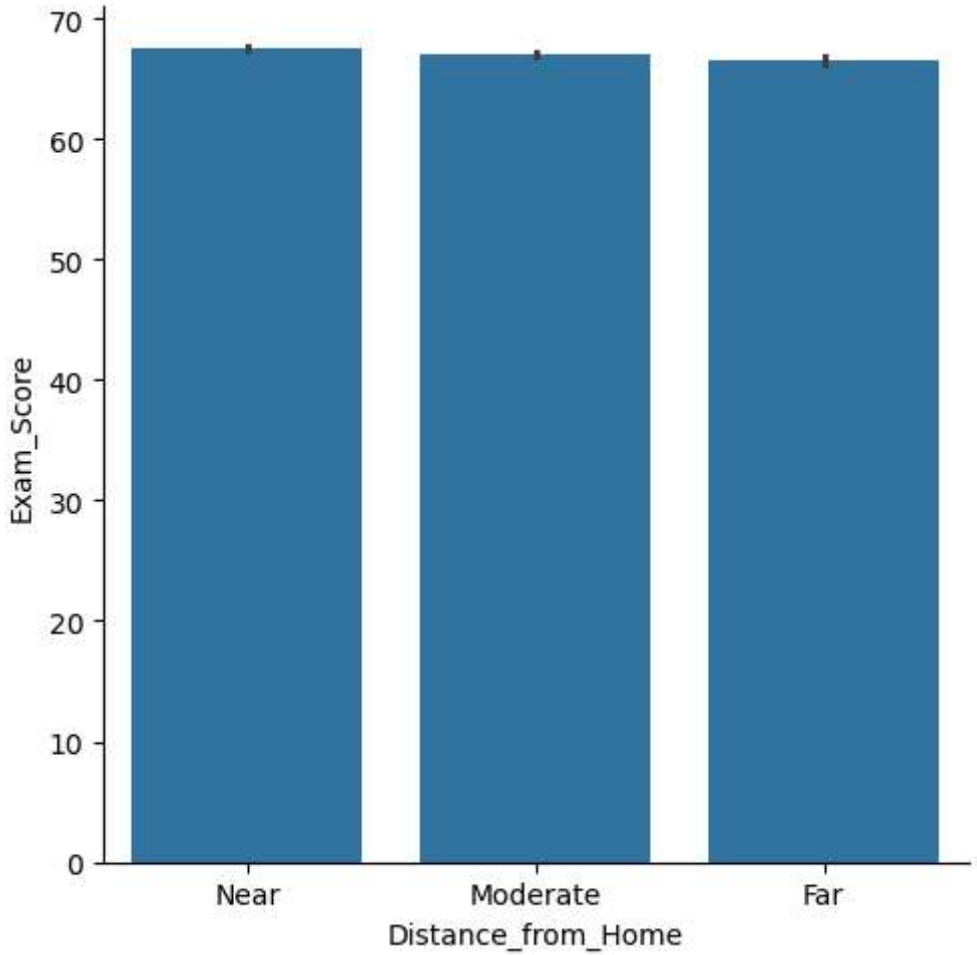
Out[331…
```
Distance_from_Home
Far          99
Moderate    101
Near        100
Name: Exam_Score, dtype: int64
```

In [333…
```
df.groupby("Distance_from_Home")["Exam_Score"].mean()
```

Out[333…
```
Distance_from_Home
Far         66.457447
Moderate    66.981481
Near        67.512101
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by Distance_from_Home
# so that there is no effect on exam_score  due to the Distance_from_Home
```

In [335…
```
sns.catplot(x="Distance_from_Home",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [ ]:

In [337…
```
df.groupby("Gender")["Exam_Score"].max()
```

Out[337…
```
Gender
Female    101
Male       99
Name: Exam_Score, dtype: int64
```

In [339…
```
df.groupby("Gender")["Exam_Score"].min()
```

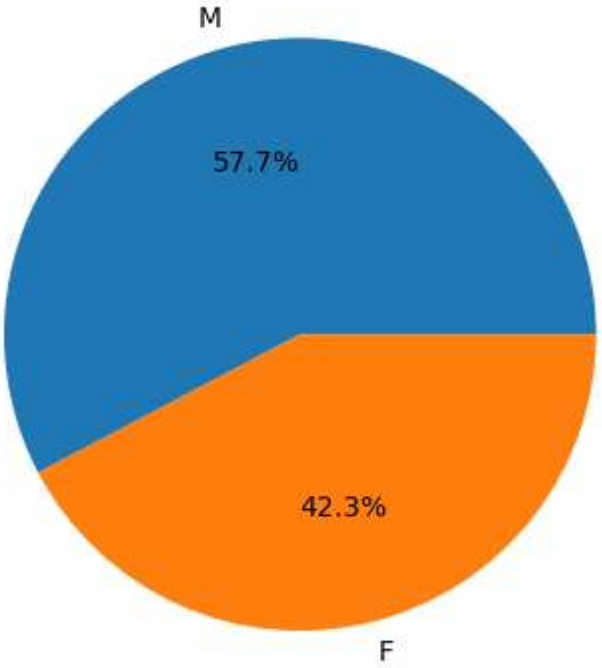
Out[339…
```
Gender
Female    57
Male      55
Name: Exam_Score, dtype: int64
```

In [341…
```
df.groupby("Gender")["Exam_Score"].mean()
```

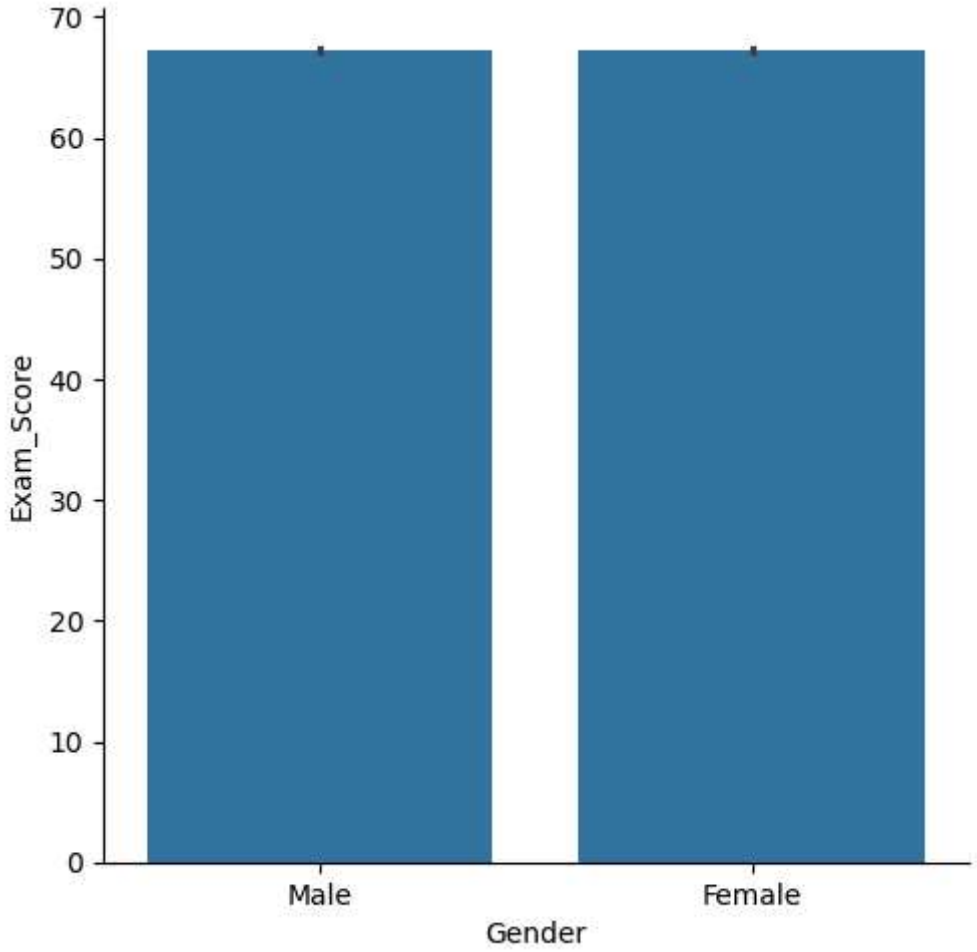
Out[341…
```
Gender
Female    67.244898
Male      67.228894
Name: Exam_Score, dtype: float64
```

In [ ]:
```
# the mean,minmum,maximum exam_score is not effected by gender
# so that there is no effect on exam_score  due to the gender
```

In [361…
```
plt.pie(df["Gender"].value_counts(),labels=["M","F"],autopct="%0.1f%%")
plt.show()
```



In [343…
```
sns.catplot(x="Gender",y="Exam_Score",data=df,kind="bar")
plt.show()
```



In [11]:
```
sns.heatmap(moderate_effected_columns,annot=True)
plt.xticks(rotation=15)   # Rotate x-axis tick labels
plt.yticks(rotation=75)
plt.savefig("StudiedAttendanceSleepTutoringExam")
plt.show()
```

In [ ]:
```
non_effected_columns=["Parental_Involvement","Access_to_Resources","Extracurricular_Activities",
                      "Previous_Scores","Motivation_Level","Internet_Access","Family_Income",
                       "Teacher_Quality","School_Type","Peer_Influence","Physical_Activity",
                       "Learning_Disabilities","Parental_Education_Level","Distance_from_Home","Gender"]
```

In [ ]:
```
moderate_effected_columns=df[["Studied_Hours","Attendance","Sleep_Hours","Tutoring_Sessions","Exam_Score"]].corr()
moderate_effected_columns
```

In [ ]:
```
#    Moderately influential factors for better exam scores are:

#           Attendance (most impactful)
#           Studied_Hours

#    Less influential factors:

#            Tutoring_Sessions (minor effect)

#   No significant effect:
```

```
#            Sleep_Hours
```

In [ ]:
```
By MANISH KUMAR
```